

**UNIVERSIDADE FEEVALE
CIÊNCIA DA COMPUTAÇÃO**

JOSUÉ HENRIQUE BECKER SCHWARTZHAUPT

**PROJETO FINAL - QUIOSQUE DE AUTOATENDIMENTO
UTILIZANDO JAVA FX**

Professor: Rodrigo Rafael Villarreal Goulart

Novo Hamburgo

2026/1

SUMÁRIO

1 INTRODUÇÃO.....	1
2 DIAGRAMA DE CLASSES UML.....	2
3 DESCRIÇÃO DOS PRINCIPAIS COMPONENTES.....	4
3.1 Classe Sistema.....	4
3.2 Classe Pedido.....	4
3.3 Classe Abstrata Item.....	4
3.4 Classe Lanche.....	4
3.5 Classe Bebida.....	4
3.6 Classe Sobremesa.....	5
3.7 Classe Cardapio.....	5
3.8 Enumerações Tamanho e Sabor.....	5
3.9 Classe TelaGeralController.....	5
4 DECISÕES DE MODELAGEM.....	6
4.1 Organização em Camadas e Pacotes.....	6
4.2 Gerenciamento Centralizado do Sistema.....	7
4.3 Uso de Herança e Polimorfismo.....	7
4.4 Utilização de Coleções do Tipo Map.....	7
4.5 Implementação de equals() e hashCode().....	7
4.6 Uso da Interface Comparable.....	8
4.9 Uso de Generics.....	8
4.10 Atualização Centralizada da Interface.....	8
4.11 Encapsulamento e Imutabilidade.....	9
5 FUNCIONAMENTO.....	10
6 CONCLUSÃO.....	12

1 INTRODUÇÃO

O projeto tem como objetivo o desenvolvimento de um sistema de autoatendimento para lanchonetes e restaurantes, simulando o funcionamento de um totem utilizado por clientes para realizar pedidos de forma rápida e autônoma. A aplicação foi desenvolvida em Java utilizando a biblioteca JavaFX para a construção da interface gráfica, permitindo a interação entre clientes e estabelecimento em um único ambiente.

A proposta consiste em uma tela dividida em duas áreas principais. No lado esquerdo encontra-se a interface destinada ao cliente, onde é possível visualizar os produtos disponíveis, adicionar ou remover itens do pedido, acompanhar o valor total da compra e confirmar ou cancelar o atendimento. No lado direito está a área destinada ao gerenciamento dos pedidos pelo estabelecimento, permitindo visualizar a lista de atendimentos realizados, consultar informações detalhadas de cada pedido, acompanhar seu status e atualizar seu progresso durante o processo de preparação.

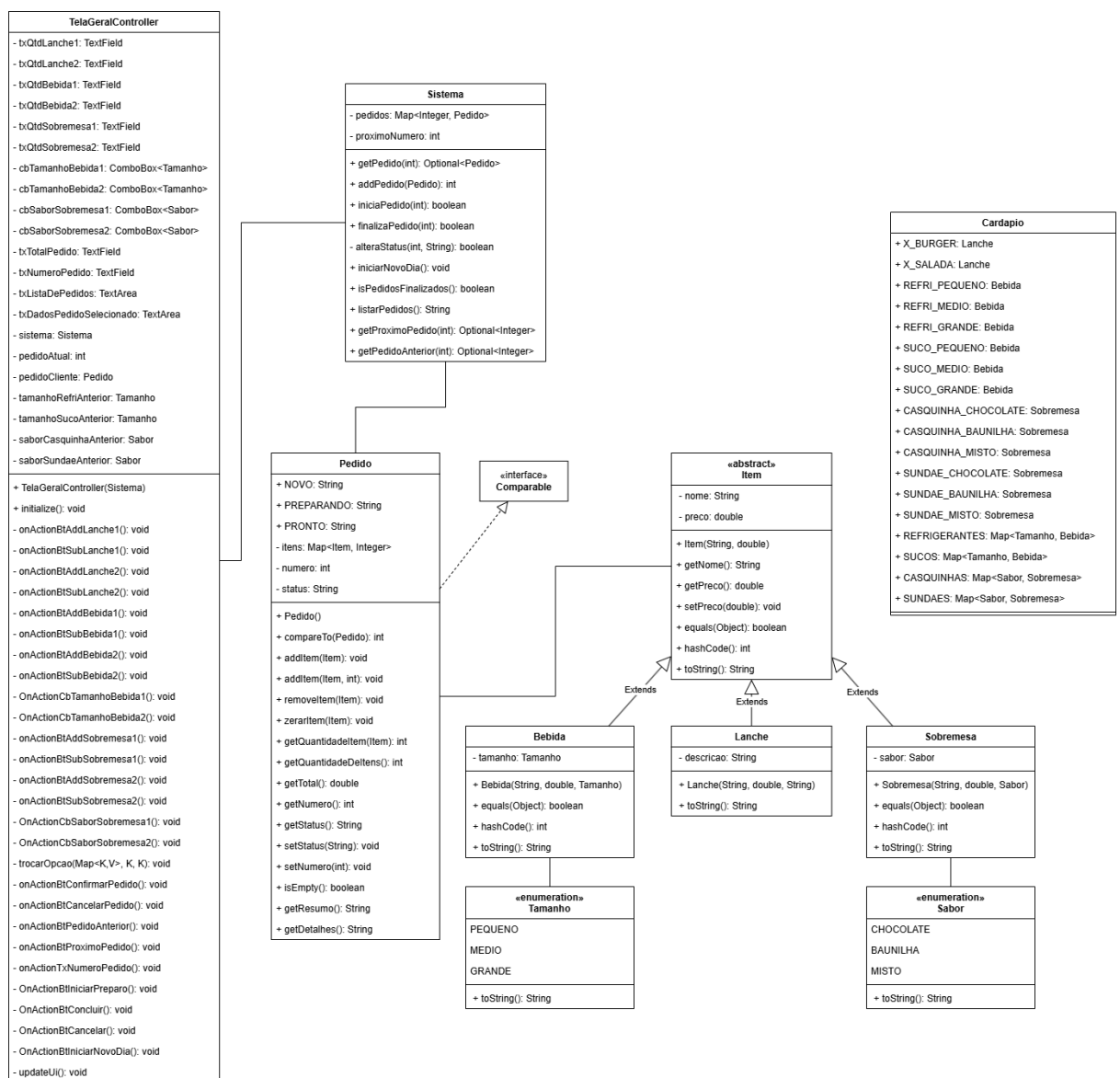
Além de reproduzir uma situação real encontrada em restaurantes e lancherias, o projeto tem como finalidade consolidar os conhecimentos adquiridos ao longo da disciplina de Programação I. Para isso, foram aplicados conceitos fundamentais como encapsulamento, herança, polimorfismo, classes abstratas, interfaces, coleções e organização modular do código. O desenvolvimento exigiu a criação de toda a modelagem do sistema desde o início, envolvendo pesquisa, análise de requisitos, abstração de entidades do domínio e elaboração de um diagrama UML que serviu como base para a implementação.

Dessa forma, o trabalho representa não apenas a construção de uma aplicação funcional, mas também uma oportunidade de aprofundar o conhecimento em análise, modelagem e desenvolvimento de software orientado a objetos, colocando em prática técnicas utilizadas no desenvolvimento de sistemas reais e adquirindo experiência no planejamento e estruturação de projetos de maior porte.

2 DIAGRAMA DE CLASSES UML

Para auxiliar no desenvolvimento do sistema foi elaborado um Diagrama de Classes UML, representando a estrutura da aplicação, os relacionamentos entre as classes e a divisão das responsabilidades de cada componente.

Figura 1 – Diagrama de Classes UML do sistema



O diagrama foi organizado em três camadas principais: Interface, Controle e Modelo. A camada de Interface é representada pela classe `TelaGeralController`, responsável pela interação com o usuário. A camada de Controle é composta pela

classe *Sistema*, que gerencia os pedidos e as regras de negócio da aplicação. Já a camada de Modelo reúne as classes responsáveis por representar os dados do sistema, como *Pedido*, *Item*, *Lanche*, *Bebida* e *Sobremesa*.

Durante a modelagem foram aplicados conceitos fundamentais da Programação Orientada a Objetos. A classe abstrata *Item* foi criada para reunir características comuns dos produtos, permitindo que as classes derivadas reutilizem seus atributos e métodos. As enumerações *Tamanho* e *Sabor* foram utilizadas para representar opções fixas do sistema, enquanto a interface *Comparable* foi empregada para permitir a ordenação dos pedidos.

Além disso, foram utilizadas coleções para armazenar os pedidos e os itens selecionados pelos clientes, proporcionando maior flexibilidade e eficiência no gerenciamento das informações.

A modelagem serviu como base para a implementação do sistema, auxiliando na organização do código e na aplicação adequada dos conceitos estudados ao longo da disciplina.

3 DESCRIÇÃO DOS PRINCIPAIS COMPONENTES

3.1 Classe Sistema

A classe *Sistema* é responsável pelo gerenciamento geral da aplicação. Ela centraliza o armazenamento dos pedidos realizados pelos clientes e disponibiliza operações para cadastro, consulta e atualização dos atendimentos.

Além disso, essa classe é responsável pela geração automática dos números de pedido, pelo controle dos status dos atendimentos e pela inicialização de um novo ciclo de operação da lanchonete.

3.2 Classe Pedido

A classe *Pedido* representa um atendimento realizado por um cliente. Cada objeto dessa classe armazena as informações relacionadas ao pedido, incluindo os itens selecionados, o número identificador e o status atual do atendimento.

Também é responsável pelo cálculo do valor total da compra, pela geração de resumos para exibição na interface e pelo controle dos produtos adicionados ou removidos pelo cliente.

3.3 Classe Abstrata Item

A classe abstrata *Item* representa a estrutura básica de qualquer produto comercializado pelo estabelecimento. Ela concentra atributos comuns, como nome e preço, além de comportamentos compartilhados entre os diferentes tipos de produtos.

A utilização de uma classe abstrata permitiu evitar duplicação de código e facilitar a criação de novas categorias de produtos futuramente.

3.4 Classe Lanche

A classe *Lanche* herda as características da classe *Item* e representa os sanduíches disponíveis para venda. Além dos atributos herdados, possui uma descrição utilizada para apresentar informações adicionais sobre sua composição.

3.5 Classe Bebida

A classe *Bebida* também deriva da classe *Item* e representa os produtos líquidos comercializados pelo estabelecimento. Cada bebida possui uma informação adicional referente ao seu tamanho, permitindo a existência de diferentes versões do mesmo produto.

3.6 Classe Sobremesa

A classe *Sobremesa* representa os produtos destinados à sobremesa, como casquinhas e sundaes. Assim como ocorre nas bebidas, cada sobremesa pode possuir diferentes variações, identificadas por seu sabor.

3.7 Classe Cardapio

A classe *Cardapio* funciona como um catálogo central dos produtos disponíveis para venda. Nela são definidos todos os itens que podem ser selecionados pelos clientes durante a utilização do sistema.

A centralização dessas informações facilita a manutenção do cardápio e reduz a necessidade de criação repetida dos mesmos objetos ao longo da aplicação.

3.8 Enumerações Tamanho e Sabor

As enumerações *Tamanho* e *Sabor* foram criadas para representar conjuntos fixos de opções disponíveis para determinados produtos.

Sua utilização garante maior consistência dos dados e simplifica o preenchimento dos componentes de seleção presentes na interface gráfica.

3.9 Classe TelaGeralController

A classe *TelaGeralController* é responsável por intermediar a comunicação entre a interface gráfica e as classes de negócio do sistema. Ela recebe as ações realizadas pelos usuários, executa as operações necessárias e atualiza os elementos visuais da aplicação.

Além disso, essa classe controla tanto a área de autoatendimento do cliente quanto a área de gerenciamento dos pedidos, permitindo que ambas permaneçam sincronizadas durante a execução do sistema.

4 DECISÕES DE MODELAGEM

Durante o desenvolvimento do sistema, diversas decisões de modelagem foram tomadas com o objetivo de aplicar os conceitos de Programação Orientada a Objetos e tornar o código mais organizado, reutilizável e de fácil manutenção.

4.1 Organização em Camadas e Pacotes

O projeto foi organizado seguindo o padrão de arquitetura em camadas, separando as responsabilidades em pacotes distintos dentro de **com.josuesch**:

- **com.josuesch** - camada de aplicação. Contém a classe App, responsável por inicializar o ambiente JavaFX, carregar a interface gráfica e injetar as dependências necessárias no controlador.
- **com.josuesch.controller** - camada de controle. Contém a classe TelaGeralController, que faz a ligação entre a interface gráfica (arquivo FXML) e o modelo, respondendo às ações do usuário e atualizando a tela.
- **com.josuesch.model** - camada de modelo. Contém as classes Sistema, Pedido e Cardapio, que representam as regras de negócio e o gerenciamento dos dados da aplicação.
- **com.josuesch.model.item** - subpacote do modelo responsável pela hierarquia de itens do cardápio. Contém a classe abstrata Item e suas subclasses concretas Lanche, Bebida e Sobremesa.
- **com.josuesch.model.enums** - subpacote do modelo que agrupa os tipos enumerados utilizados para representar variações de itens: Tamanho (PEQUENO, MEDIO, GRANDE) e Sabor (CHOCOLATE, BAUNILHA, MISTO).

Essa separação permite que a camada de modelo seja completamente independente da interface gráfica, tornando possível reutilizar as classes de negócio em outros contextos, como um sistema para o Restaurante Universitário ou qualquer outra lancheria, sem necessidade de alterações.

4.2 Gerenciamento Centralizado do Sistema

Foi adotada uma única instância da classe *Sistema*, criada na classe principal da aplicação e compartilhada com o controlador da interface. Essa abordagem garante que todos os componentes manipulem os mesmos dados, evitando inconsistências e simplificando o gerenciamento dos pedidos.

4.3 Uso de Herança e Polimorfismo

Os produtos foram modelados a partir da classe abstrata *Item*, que reúne características comuns como nome e preço. As classes *Lanche*, *Bebida* e *Sobremesa* especializam esse comportamento por meio da herança.

Essa estrutura permite que diferentes tipos de produtos sejam manipulados de forma uniforme através de referências do tipo *Item*, aplicando o conceito de polimorfismo.

4.4 Utilização de Coleções do Tipo Map

Os pedidos armazenam seus produtos utilizando uma estrutura *Map<Item, Integer>*, na qual a chave representa o produto e o valor representa sua quantidade. Essa solução foi escolhida em vez de uma lista tradicional porque evita a duplicação de objetos iguais dentro do pedido e permite acesso direto à quantidade de cada item, simplificando operações de adição, remoção e atualização.

De forma semelhante, a classe *Sistema* utiliza um *Map<Integer, Pedido>* para armazenar os pedidos cadastrados, permitindo acesso rápido através do número de identificação.

4.5 Implementação de equals() e hashCode()

Como os objetos são utilizados como chaves em estruturas do tipo *Map*, foi necessária a implementação dos métodos *equals()* e *hashCode()*.

Nas classes *Bebida* e *Sobremesa*, esses métodos consideram não apenas o nome do produto, mas também suas características específicas, como tamanho ou sabor. Dessa forma, um refrigerante pequeno e um refrigerante grande são tratados como produtos distintos dentro do sistema.

4.6 Uso da Interface Comparable

A classe *Pedido* implementa a interface *Comparable*, permitindo que os pedidos sejam ordenados automaticamente pelo número de atendimento.

Essa decisão simplifica operações de listagem e navegação entre pedidos, além de demonstrar a aplicação de contratos definidos por interfaces.

4.7 Utilização de Optional

Diversos métodos retornam objetos do tipo *Optional*, especialmente durante a busca de pedidos.

Essa abordagem foi adotada para reduzir a possibilidade de erros relacionados a referências nulas (*NullPointerException*) e tornar explícita a possibilidade de um resultado não existir.

4.8 Utilização de Streams

A API Stream foi utilizada em diversos métodos para realizar filtragens, ordenações e processamentos de coleções de forma mais declarativa.

Essa abordagem reduz a quantidade de código repetitivo e melhora a legibilidade das operações realizadas sobre os pedidos armazenados pelo sistema.

4.9 Uso de Generics

Para evitar duplicação de código durante a troca de tamanhos e sabores dos produtos, foi criado o método genérico:

```
trocarOpcao(Map<K, V>, K anterior, K atual)
```

A utilização de generics permite reutilizar a mesma lógica para diferentes tipos de mapas e produtos, mantendo segurança de tipos em tempo de compilação e reduzindo a necessidade de múltiplas implementações semelhantes.

4.10 Atualização Centralizada da Interface

Foi criado o método *updateUi()*, responsável por atualizar todos os componentes visuais da aplicação.

Essa centralização evita a repetição de código em diversos eventos da

interface e garante que todas as informações exibidas permaneçam sincronizadas com os dados do sistema.

4.11 Encapsulamento e Imutabilidade

Os atributos das classes foram declarados como privados sempre que possível, sendo acessados por meio de métodos específicos.

Além disso, a palavra-chave final foi utilizada em atributos que não devem ser alterados após sua inicialização, contribuindo para a segurança e previsibilidade do comportamento da aplicação.

5 FUNCIONAMENTO

O sistema foi desenvolvido para simular o funcionamento de um totem de autoatendimento em uma lanchonete. A aplicação apresenta uma interface dividida em duas áreas: uma destinada ao cliente e outra destinada ao gerenciamento dos pedidos pelo estabelecimento.

Inicialmente, o cliente pode visualizar os produtos disponíveis no cardápio, organizados em categorias como lanches, bebidas e sobremesas. Ao selecionar os itens desejados, o sistema atualiza automaticamente o resumo do pedido, exibindo as quantidades escolhidas e o valor total da compra.

Após concluir a seleção dos produtos, o cliente pode confirmar o pedido. Nesse momento, o sistema gera automaticamente um número de atendimento único e registra o pedido para acompanhamento pelo estabelecimento. Caso o cliente desista da compra, é possível cancelar o pedido antes da confirmação.

Na área de gerenciamento, os pedidos realizados ficam disponíveis para consulta. O responsável pelo atendimento pode navegar entre os pedidos cadastrados e acompanhar suas informações detalhadas, incluindo itens selecionados, valor total e status atual.

Cada pedido passa por um fluxo de estados que representa seu progresso dentro da lanchonete. Inicialmente, o pedido é registrado com o status **"A Preparar"**. Quando a produção é iniciada, o status é alterado para **"Preparando"**. Após a finalização, o pedido recebe o status **"Pronto"**, indicando que pode ser entregue ao cliente.

Ao final do expediente, o sistema permite iniciar um novo dia de operação. Essa funcionalidade remove todos os pedidos armazenados e reinicia a numeração dos atendimentos, garantindo que uma nova sessão de utilização possa ser iniciada de forma organizada.

A Figura 2 apresenta a interface principal da aplicação, demonstrando simultaneamente a área de autoatendimento do cliente e o painel de gerenciamento dos pedidos.

Figura 2 – Interface principal do sistema

Lista de Pedidos:

Pedido #001 | 1 item | R\$ 18,00 | A Preparar
Pedido #002 | 2 itens | R\$ 23,00 | A Preparar

Processar Pedido

Nº: 1 Anterior Proximo

Dados:

=====

PEDIDO #001

=====

Status: A Preparar
Quantidade de itens: 1
Valor total: R\$ 18,00

Iniciar Preparo Concluir Cancelar

Iniciar Novo Dia

Total: 22,00

Confirmar Cancelar

6 CONCLUSÃO

O desenvolvimento deste projeto permitiu a aplicação prática dos principais conceitos estudados ao longo da disciplina de Programação I. A partir da modelagem e implementação de um sistema de autoatendimento para lanchonetes, foi possível compreender de forma mais aprofundada a importância do planejamento, da organização do código e da correta divisão de responsabilidades entre as classes.

Durante o desenvolvimento foram utilizados conceitos como encapsulamento, herança, polimorfismo, abstração, interfaces e coleções, além de recursos da linguagem Java que contribuíram para a construção de uma solução mais robusta e organizada. A elaboração do diagrama UML antes da implementação também auxiliou na definição da estrutura do sistema e na identificação dos relacionamentos entre seus componentes.

O resultado obtido foi uma aplicação funcional, capaz de simular o processo de realização e gerenciamento de pedidos em um ambiente de autoatendimento. Além de atender aos requisitos propostos, o projeto proporcionou experiência prática em análise de requisitos, modelagem de software, desenvolvimento de interfaces gráficas com JavaFX e aplicação de boas práticas de programação.

Por fim, o trabalho contribuiu significativamente para o aprimoramento dos conhecimentos em desenvolvimento orientado a objetos, permitindo consolidar conteúdos estudados em aula e adquirir experiência na construção de um sistema completo, desde a fase de planejamento até sua implementação final.